

实验报告：语义重构与本地开发反馈

项目	内容
课程名称	系统开发工具基础
实验主题	开发环境与工具
题目编号	第6题
学号	25020007191
姓名	周御基
实验日期	2026年8月31日

一、实验目的

1. 搭建含 Python、pytest、ruff 的本地开发环境
2. 启用语言服务器（LSP），演示跳转到定义、查找引用
3. 使用"重命名符号"完成一次语义重构，保证定义与所有引用同步更新
4. 用 ruff 发现未使用 import 并删除，运行 ruff 与 pytest 保证全部通过

二、实验环境

- 操作系统：Ubuntu Linux（远程服务器 192.168.79.130）
- Python：3.13.3
- 工具：pytest 9.1.1、ruff 0.16.5、jedi（LSP 引擎）
- 远程连接：SSH（paramiko）

三、实验步骤与代码

步骤1：配置开发环境（安装 pip、pytest、ruff、LSP 引擎）

Ubuntu 25.04 默认无 pip，先安装 pip，再安装 pytest、ruff 和 jedi（LSP 引擎）：

```
cd /tmp && python3 get-pip.py --user --break-system-packages
python3 -m pip install --user --break-system-packages pytest ruff python-lsp-server
python3 -m pytest --version
python3 -m ruff --version
python3 -m pylsp --version
```

输出结果:

```
pip 26.2.1 ...
Successfully installed pytest-9.1.1 ruff-0.16.5
pytest 9.1.1
ruff 0.16.5
__main__.py v1.15.0
```

注: 虚拟机未装 `python3-venv` 且用户无 `sudo` 权限, 故采用 `--user --break-system-packages` 安装到用户目录 (本实验环境为教学用虚拟机)。

步骤2: 创建三个给定的 Python 文件

创建 `~/q06` 下的 `math_utils.py`、`app.py`、`test_math_utils.py` :

```
mkdir -p ~/q06
cat > ~/q06/math_utils.py << 'EOF'
def total_price(price: float, count: int) -> float:
    return price * count
EOF
cat > ~/q06/app.py << 'EOF'
from math_utils import total_price
print(total_price(12.5, 4))
EOF
cat > ~/q06/test_math_utils.py << 'EOF'
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF
```

输出结果:

(三个文件创建成功)

步骤3：在重构前运行 pytest 与 ruff，作为基准

```
cd ~/q06 && pytest test_math_utils.py -v
```

输出结果：

```
test_math_utils.py::test_total_price PASSED
```

```
===== 1 passed in 0.02s =====
```

```
ruff check .
```

输出结果（节选）：

```
I001 [*] Import block is un-sorted or un-formatted → app.py / test_math_utils.py
```

测试通过；ruff 提示 `app.py`、`test_math_utils.py` 的 import 块需要空行分隔（I001）。

步骤4：跳转到定义与查找引用

使用语言服务器引擎 jedi 从 `app.py` 中定位 `total_price` 的定义，并查找其在项目中的全部引用：

```
# jump-to-def: 定位 app.py 第1行 'total_price' 的定义
defs = jedi.Script(app_src, path=app).infer(1, 27)
➡ /home/joe/q06/math_utils.py:1 total_price

# find-references: 在 math_utils.py 定义处查找所有引用
refs = jedi.Script(mu_src, path=math_utils).get_references(1, 4)
➡ /home/joe/q06/app.py:1:23 total_price
/home/joe/q06/app.py:2:6 total_price
/home/joe/q06/math_utils.py:1:4 total_price
/home/joe/q06/test_math_utils.py:1:23 total_price
/home/joe/q06/test_math_utils.py:3:11 total_price
```

- **跳转到定义：**从 `app.py` 第 1 行的 `total_price` 跳转到 `math_utils.py:1` 的函数定义处。
- **查找引用：**找到项目中所有 5 处引用（`app.py` 2 处、`math_utils.py` 定义 1 处、`test_math_utils.py` 2 处）。

步骤5：使用"重命名符号"完成语义重构

把 `total_price` 重命名为 `calculate_total`（定义 + 所有代码引用同步改变）：

```
for path, line, col in [("math_utils.py", 1, 4), ("app.py", 1, 27),
                        ("test_math_utils.py", 1, 31)]:
    script = jedi.Script(src, path=path)
    script.rename(line, col, new_name="calculate_total").apply()
```

重构后的文件:

```
# math_utils.py
def calculate_total(price: float, count: int) → float:
    return price * count

# app.py
from math_utils import calculate_total

print(calculate_total(12.5, 4))

# test_math_utils.py
from math_utils import calculate_total

def test_total_price():
    assert calculate_total(12.5, 4) == 50.0
```

步骤6: 临时加入未使用 import os, 用 ruff 发现并删除

把 app.py 第 1 行临时改成 `import os` + 原内容, 运行 ruff:

```
ruff check .
```

输出结果:

```
F401 [*] os imported but unused
→ app.py:1:8
1 | import os
```

ruff 指出 `os` 被导入但未使用 (F401)。删除该行后再次运行:

```
ruff check .
All checks passed!
```

步骤7: 运行 ruff 与 pytest 验证全部通过

```
ruff check .  
pytest test_math_utils.py -v
```

输出结果:

```
All checks passed!  
test_math_utils.py::test_total_price PASSED  
===== 1 passed in 0.04s =====
```

四、实验结果

4.1 结果展示

- 跳转定义、查找引用、重命名符号三项 LSP 功能演示成功
- `total_price` → `calculate_total` 重构后, 定义与全部 5 处引用同步更新
- 临时加入的 `import os` 被 ruff 以 F401 检出, 删除后恢复干净
- 最终 `ruff check .` → `All checks passed!`, `pytest` → `1 passed`

4.2 结果分析

1. 语义重构 (rename symbol) 与文本替换 (sed/全局替换) 不同: 它基于语法树理解符号作用域, 只改同名符号对应的定义与引用, 不会误改同名字符串。
2. LSP 的"跳转定义/查找引用/重命名"让大型代码库中的符号级操作可靠、可追踪。
3. 语言服务器引擎 jedi 提供了与 pylsp 一致的语义能力, 可在命令行下复现 IDE 的核心操作。
4. 开发反馈闭环: `ruff` (静态检查) 发现未使用 import → 手动删除 → 再次 `ruff` 通过 → `pytest` 通过, 形成本地质量门禁。

五、实验总结

通过本次实验, 我掌握了以下技能:

1. 在命令行搭建 Python + pytest + ruff + LSP 的开发环境
2. 使用语言服务器实现跳转定义、查找引用
3. 用"重命名符号"完成安全、一致的语义重构
4. 用 ruff 发现未使用 import 等 lint 问题并修复, 运行 ruff 与 pytest 保证全部通过

实验中的脚本文件:

1. `Scripts/q06/math_utils.py`
2. `Scripts/q06/app.py`
3. `Scripts/q06/test_math_utils.py`
4. `Scripts/q06/q06_demo.py`